EduTrack

Phase 2: UML Modelling Documentation

Teacher Management & Learner Portal System

React | TypeScript | Supabase | Vercel

Includes: Class • Sequence • State • Activity • Component • Deployment Diagrams

Table of Contents

1. **1. Class Diagram**
   Defines all domain entities. Each class box shows the class name in a coloured header, attributes in the middle section, and metho…
2. **2. Sequence Diagram**
   Visualises four time-ordered interaction flows: (A) Teacher login via Supabase GoTrue, (B) Learner login via email + student numbe…
3. **3. State Diagram**
   Describes the Learner record lifecycle: Not Registered → Registered → Active → performance sub-states (At Risk / Performing / Top …
4. **4. Activity Diagram**
   Shows the add-learner workflow across three swim lanes: Teacher/Browser, EduTrack Backend (Supabase), and Notification System. Dec…
5. **5. Component Diagram**
   Depicts logical software components and their dependencies. Three packages group components: Frontend (React/Vite), Backend (Supab…
6. **6. Deployment Diagram**
   Maps the physical production topology. 3-D node boxes represent: Developer Machine, GitHub Repository, Vercel CDN/Edge, User Brows…

Stack: React 18 • TypeScript • Vite • Tailwind CSS • Supabase (PostgreSQL + GoTrue Auth + Storage) • Vercel (SPA deployment)

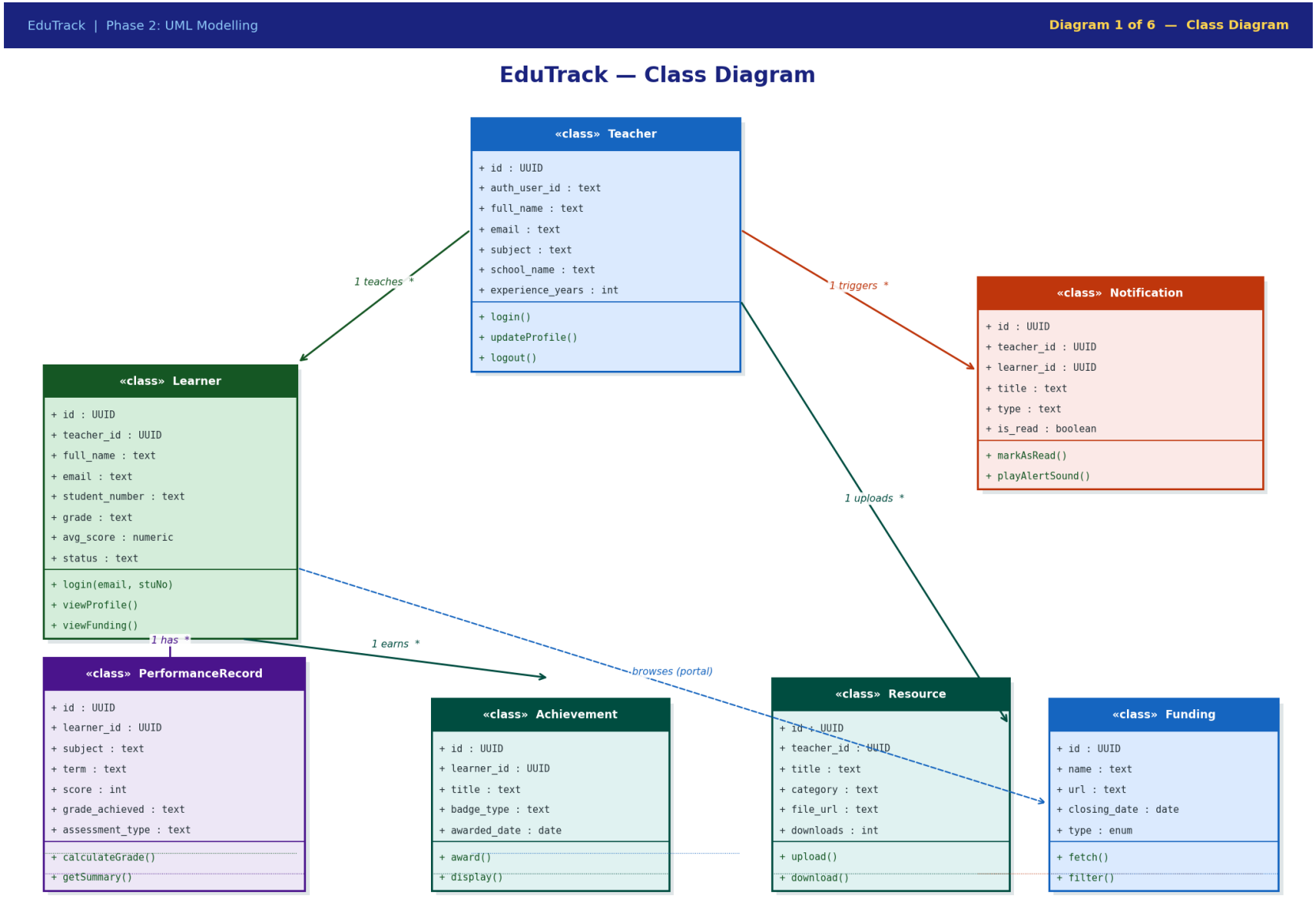
Auth: Teachers via Supabase GoTrue | Learners via email + student_number lookup (no Supabase Auth account required)

Diagram 1 of 6

1. Class Diagram

Defines all domain entities. Each class box shows the class name in a coloured header, attributes in the middle section, and methods at the bottom. Associations carry multiplicity labels (1 Teacher teaches * Learners). Dashed arrows denote dependencies (Learner browses Funding read-only).

Key Points: Teacher is root aggregate. Learner links to Teacher, PerformanceRecord, Achievement, and Notification. Funding is a separate external Supabase project.



Class Diagram

Figure 1: 1. Class Diagram

Diagram 2 of 6

2. Sequence Diagram

Visualises four time-ordered interaction flows: (A) Teacher login via Supabase GoTrue, (B) Learner login via email + student number lookup, (C) Teacher adds an at-risk learner — triggers alert sound and notification, (D) Top-performer detection fires an achievement notification.

Key Points: Activation boxes show processing periods. Dashed return arrows carry response values. NotificationManager acts asynchronously after DB insert. Section D runs when score >= 90.

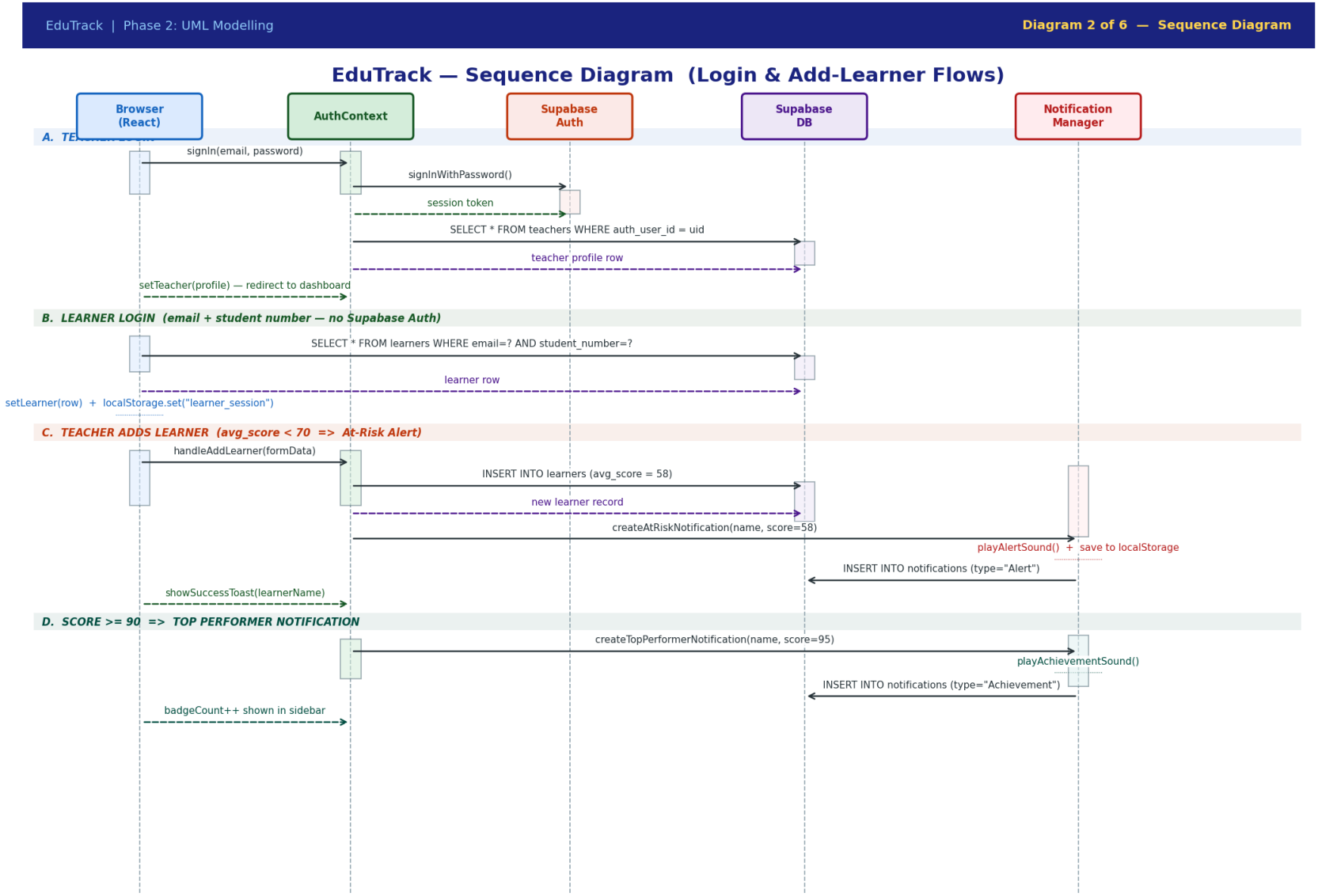


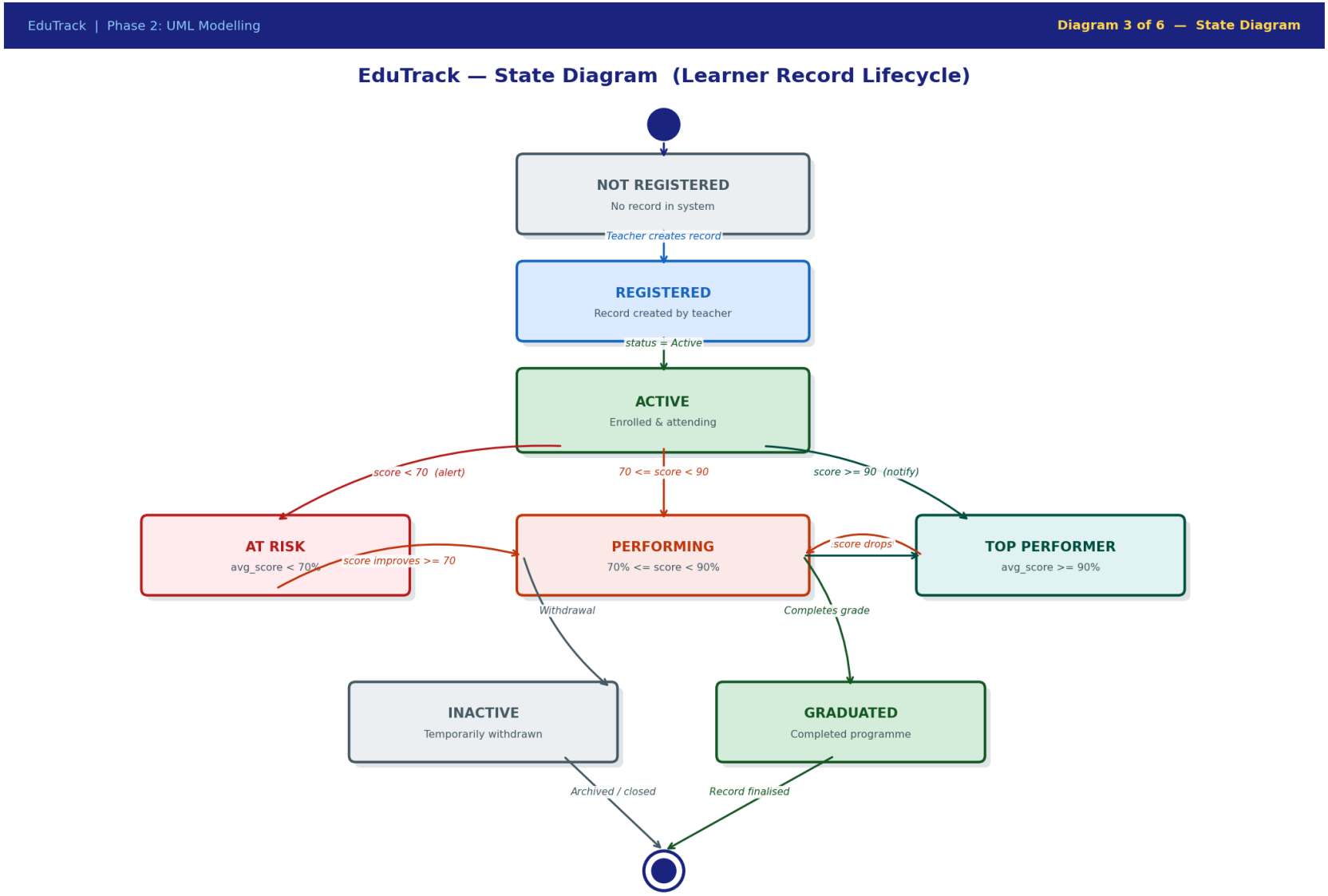
Figure 2: 2. Sequence Diagram

Diagram 3 of 6

3. State Diagram

Describes the Learner record lifecycle: Not Registered → Registered → Active → performance sub-states (At Risk / Performing / Top Performer) → terminal states (Inactive or Graduated). Guard conditions enforce business rules — avg_score < 70 triggers At Risk state and fires an alert.

Key Points: Filled black circle = initial state. Double-ringed circle = final state. Horizontal arrows between sub-states show score recovery. States map directly to the Supabase status field.



State Diagram

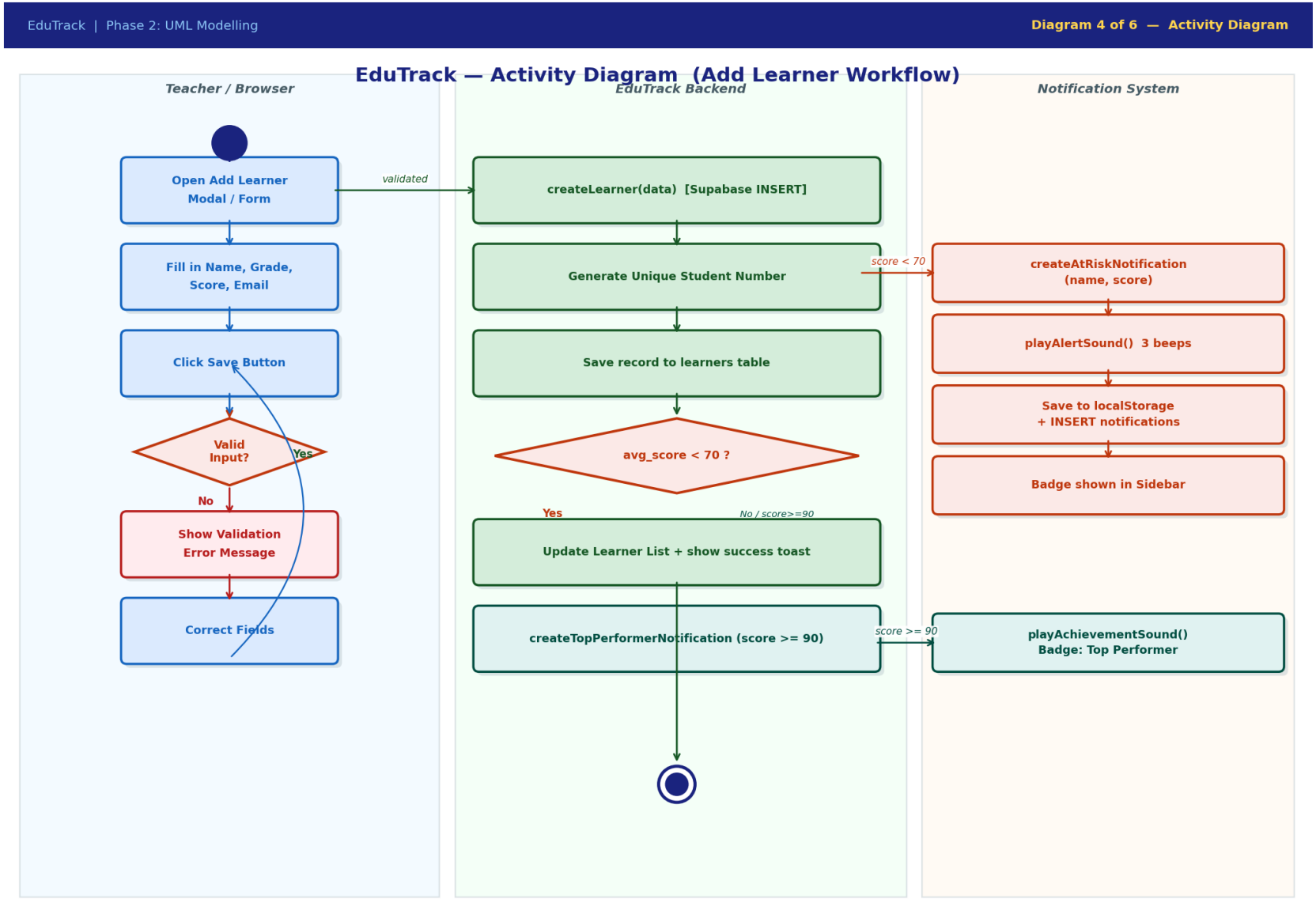
Figure 3: 3. State Diagram

Diagram 4 of 6

4. Activity Diagram

Shows the add-learner workflow across three swim lanes: Teacher/Browser, EduTrack Backend (Supabase), and Notification System. Decision diamonds handle input validation and score-threshold branching. At-risk and top-performer notification paths can execute concurrently.

Key Points: Validation failure loops back to the form. Backend INSERT evaluates score: score < 70 → orange at-risk lane; score >= 90 → teal achievement lane. Flow ends after learner list refreshes.



Activity Diagram

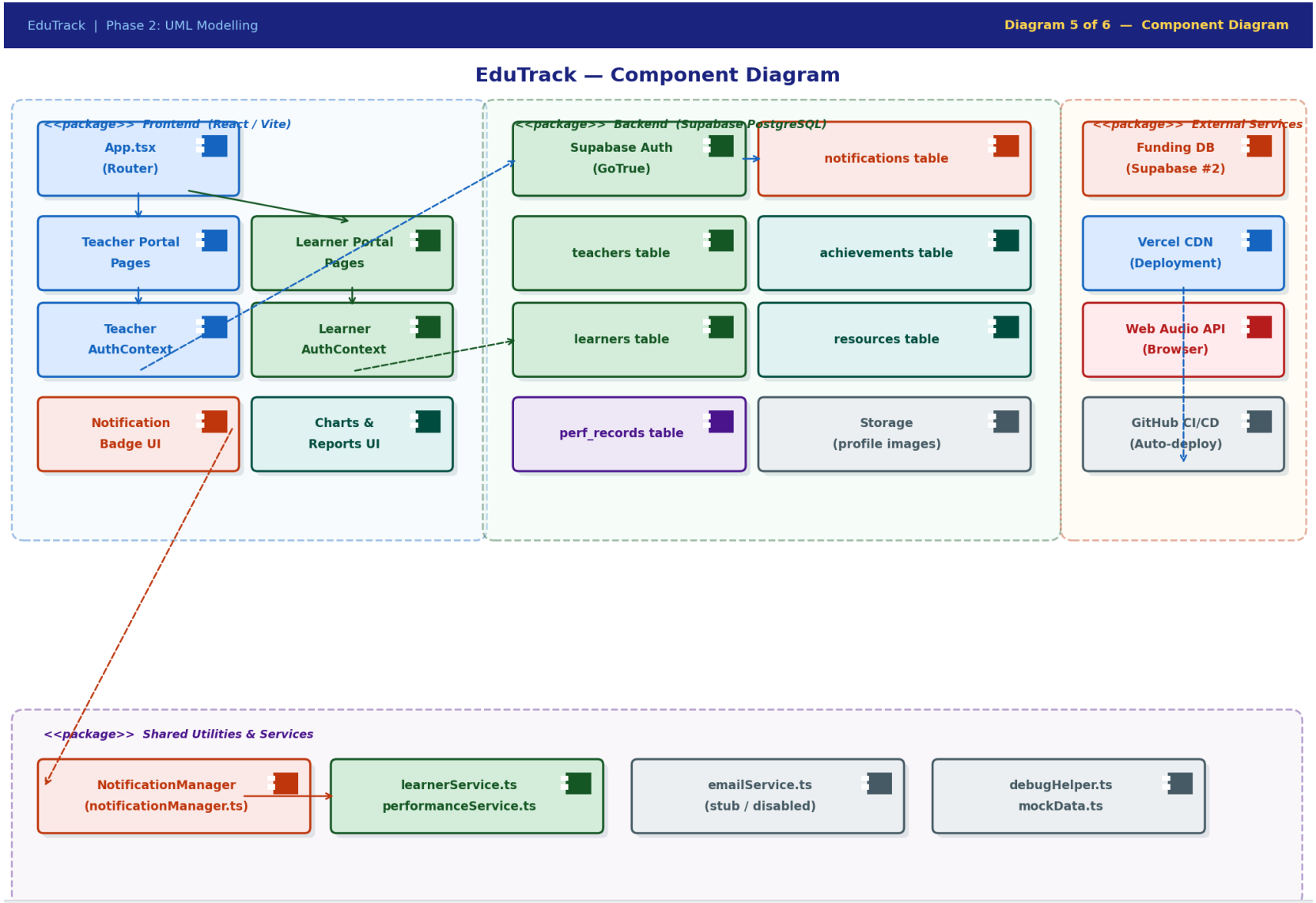
Figure 4: 4. Activity Diagram

Diagram 5 of 6

5. Component Diagram

Depicts logical software components and their dependencies. Three packages group components: Frontend (React/Vite), Backend (Supabase PostgreSQL), and External Services. A shared utilities layer sits underneath all packages. Lollipop icons denote the UML component stereotype.

Key Points: App.tsx delegates to Teacher Portal and Learner Portal. Each portal has its own AuthContext. NotificationManager bridges portal and DB. Funding DB is separate Supabase project — read-only access.



Component Diagram

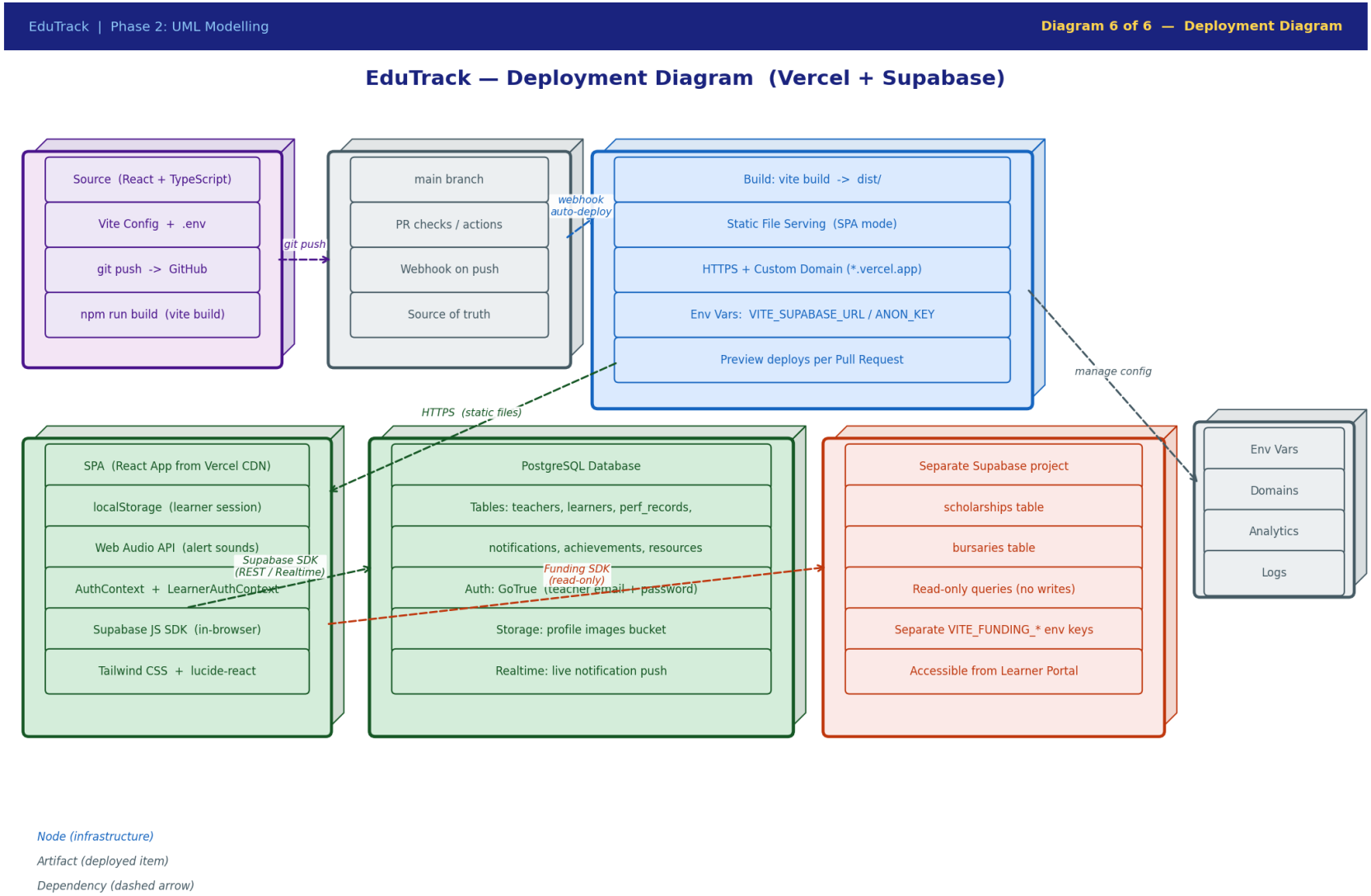
Figure 5: 5. Component Diagram

Diagram 6 of 6

6. Deployment Diagram

Maps the physical production topology. 3-D node boxes represent: Developer Machine, GitHub Repository, Vercel CDN/Edge, User Browser, Supabase Main (DB + Auth + Storage), Supabase Funding (separate project), and Vercel Dashboard. Artifacts inside each node show what is deployed.

Key Points: CI/CD: git push → GitHub webhook → Vercel auto-build (vite build) → dist/ served as static SPA over HTTPS. Env vars (VITE_SUPABASE_URL, ANON_KEY, VITE_FUNDING_*) stored in Vercel Dashboard only.



Deployment Diagram

Figure 6: 6. Deployment Diagram

Document Summary

#	Diagram	Primary Purpose	Stakeholder
1	1. Class Diagram	Defines all domain entities.	Developer / Architect
2	2. Sequence Diagram	Visualises four time-ordered interaction flows: (A) Teacher login via Supabase GoTrue, (B) Learner login via email + student ID lookup, (C) Teacher adds an at-	Developer / QA
3	3. State Diagram	Describes the Learner record lifecycle: Not Registered → Registered → Active → performance sub-states (At Risk / Performing / Top Performer) → terminal states (I	Analyst / Developer
4	4. Activity Diagram	Shows the add-learner workflow across three swim lanes: Teacher/Browser, EduTrack Backend (Supabase), and Notification System.	Business Analyst
5	5. Component Diagram	Depicts logical software components and their dependencies.	Architect / DevOps
6	6. Deployment Diagram	Maps the physical production topology.	DevOps / Client